

BC/1 - Binary Calculator Project Transport

Version 1.0 | Standalone wire specification | Page 1 of 2

1. Scope and connection rules

BC/1 transfers files using HTTP-like methods, statuses, and headers over TCP. It is not HTTP/2. A client MUST open one connection per invocation and reuse it for all requests to the same host/port; no reconnect or redirect is permitted. The server MUST answer requests in arrival order and keep the connection open after each complete response, including recoverable errors. Pipelining is permitted; multiplexing is not. The default port is 9000. Integers are unsigned and big-endian. Lengths count bytes, never characters. Text is strict UTF-8 unless further restricted below. No handshake or connection preface precedes the first frame.

2. Fixed frame header - exactly 12 bytes

- Offsets 0-1: magic, 2 bytes, hexadecimal 42 43 (ASCII BC).
- Offset 2: version, u8, MUST equal 1.
- Offset 3: type, u8; 1 = REQUEST, 2 = RESPONSE; all other values are unknown extensions.
- Offsets 4-7: payload length, u32, excludes the 12-byte header. Maximum accepted length: 8,388,608 bytes (8 MiB).
- Offsets 8-11: request ID, u32. A request MUST use a nonzero ID; each outstanding request MUST have a distinct ID. A response echoes its request ID. Zero is reserved for errors whose request ID cannot be recovered.

A receiver MUST read exactly 12 header bytes and then exactly the advertised payload bytes before parsing another frame. TCP reads may split or combine frames. A receiver meeting a frame type it does not know MUST skip it cleanly: consume its entire bounded payload, send no reply, and continue. Unknown frames do not complete an outstanding request. Magic, version, and size validation still apply to unknown types.

3. Request and response payloads

REQUEST: method_length:u8, method:bytes[method_length], path_length:u16, path:bytes[path_length], header_block. No request body or trailing bytes are allowed. Method MUST be a nonempty uppercase ASCII letter sequence; only GET is implemented (other well-formed methods return 405). Path MUST start with '/', contain no whitespace/control characters, and be a UTF-8 origin-form URI path with optional query. Fragments and authority components are invalid. The host header MUST be present and nonempty.

RESPONSE: status:u16, header_block, body:all remaining payload bytes. Status MUST be 100-599. The content-length header is required and MUST be the canonical decimal length of the body (zero is '0'; no leading zeros). Body bytes are opaque and may include NUL or non-UTF-8 data. Clients MUST reject mismatched response IDs, unexpected known frame types, invalid statuses, and mismatched content lengths. Unknown types are skipped in both directions.

4. Header block encoding

A block starts with count:u8 (0-64). Each entry is name_id:u8, then value_length:u16 and value:bytes[value_length]. If name_id is 0, insert name_length:u16 and name:bytes[name_length] immediately before value_length. Nonzero IDs MUST be 1-10 from the table below. No dynamic table, Huffman coding, or compression state exists. Unknown names MUST use literal ID 0; unassigned nonzero IDs are malformed. Literal names may also spell indexed names.

Names MUST be nonempty lowercase ASCII tokens: a-z, 0-9, and ! # \$ % & ' * + - . ^ _ ` | ~. Values are UTF-8 with no U+0000-U+001F or U+007F. Duplicate names are invalid. Unknown literal names MUST be accepted and may be ignored. The dictionary is: 1 host; 2 user-agent; 3 accept; 4 content-type; 5 content-length; 6 server; 7 connection; 8 cache-control; 9 last-modified; 10 x-request-id.

BC/1 - Behavior, limits, and rationale

Version 1.0 | Standalone wire specification | Page 2 of 2

5. File mapping and application behavior

The server is invoked as `bserve ROOT PORT`. For GET, strip the URI query, percent-decode the path exactly once as UTF-8, and resolve it relative to ROOT. Malformed percent escapes, NUL, backslash, colon, `'.'` path segments, or a resolved path outside ROOT MUST return 400. A symlink resolving outside ROOT is forbidden. Directories are not listed and have no implicit index; request `/index.html` explicitly. Missing paths and directories return 404. Readable regular files return 200 and their exact bytes. Other filesystem read failures return 500. Files larger than 8,384,512 bytes (8 MiB minus 4 KiB metadata reserve) return 413.

The supplied client defaults an omitted URL path to `/index.html`. The server sets content-type from the filename (application/octet-stream if unknown). It sends content-length, `server=bserve/1`, connection, `cache-control=no-store`, `x-request-id` as decimal, and last-modified for successful file reads (an HTTP-date in GMT). The client sends host (authority including port), `user-agent=bcurl/1`, and `accept=/*/*`. These ten header names form the fixed dictionary; they save repeated name bytes, not value bytes.

6. Errors, shutdown, and resource limits

A complete, bounded malformed request MUST receive status 400 with its envelope ID and leave the connection usable: examples include bad magic/version, truncated payload fields, invalid header indices, duplicate names, zero request ID, and a RESPONSE sent to the server. The entire advertised payload is consumed before the error is sent. Responses use the valid BC/1 envelope even when the input envelope is invalid. Error body wording is implementation-defined UTF-8 text; content-length still MUST match.

An advertised length above 8 MiB MUST yield 400 with `connection=close` and immediate closure; do not allocate or drain the oversized payload. If the peer ends its write side mid-header or mid-payload, send 400 with ID 0 and `connection=close` if writing is still possible, then close. EOF between frames is normal. Invalid framing received by a client is a protocol failure: close and exit 1, never reconnect. Socket I/O failure has the same outcome. These connection-fatal conditions override persistence.

The optional connection header MUST be either `keep-alive` or `close`; omission means `keep-alive`. A `close` request gets one response with `connection=close` and then the server closes. The default server inactivity timeout is 30 seconds per blocking socket operation, configurable with `--timeout`. Timeout closes silently. Implementations may close when resources are exhausted; the supplied server allows 64 concurrent clients. There is no TLS, authentication, flow-control negotiation, or automatic retry.

7. Client command contract

`bcurl [-v] host:port/path [host:port/path ...]` accepts an optional `bc://` prefix. Every URL in an invocation MUST use the same host and port. The client sends requests sequentially with IDs 1, 2, ... on its only connection and writes each response body unchanged to stdout. In verbose mode, every sent and received complete frame, including skipped unknown frames, is hex-dumped to stderr with direction and ID. Exit 0 if all responses are below 400; exit 22 if any response is 4xx/5xx; exit 1 on transport/protocol failure; exit 2 on CLI syntax errors. Application errors do not prevent later supplied URLs from being requested.

8. Why these fields and widths?

Two magic bytes identify accidental cross-protocol input; one version byte makes incompatible syntax explicit. An 8-bit type leaves 254 extension types that old peers can skip. A 32-bit length is simple to decode with common integer operations; the much smaller 8 MiB acceptance cap bounds memory. A 32-bit correlation ID associates errors and pipelined replies with requests, without introducing streams. u16 text lengths cover URI/header fields within the frame cap. A u8 count plus a 64-header cap bounds parsing work.

HTTP/2's 9-byte header uses a 24-bit length, 8-bit type, 8-bit flags, and a reserved bit plus a 31-bit stream ID. Those choices support compact framing and multiplexed streams. BC/1 deliberately uses neither flags nor stream state: one complete request or response occupies one frame. Its indexed header names and length-prefixed literals borrow two basic compression ideas without claiming HPACK compatibility. Reference: RFC 9113 sections 4.1 and 5; HTTP/1.1 persistence rationale: RFC 9112 section 9.3. The companion `annotated-hexdump.md` provides a complete captured exchange and byte offsets.